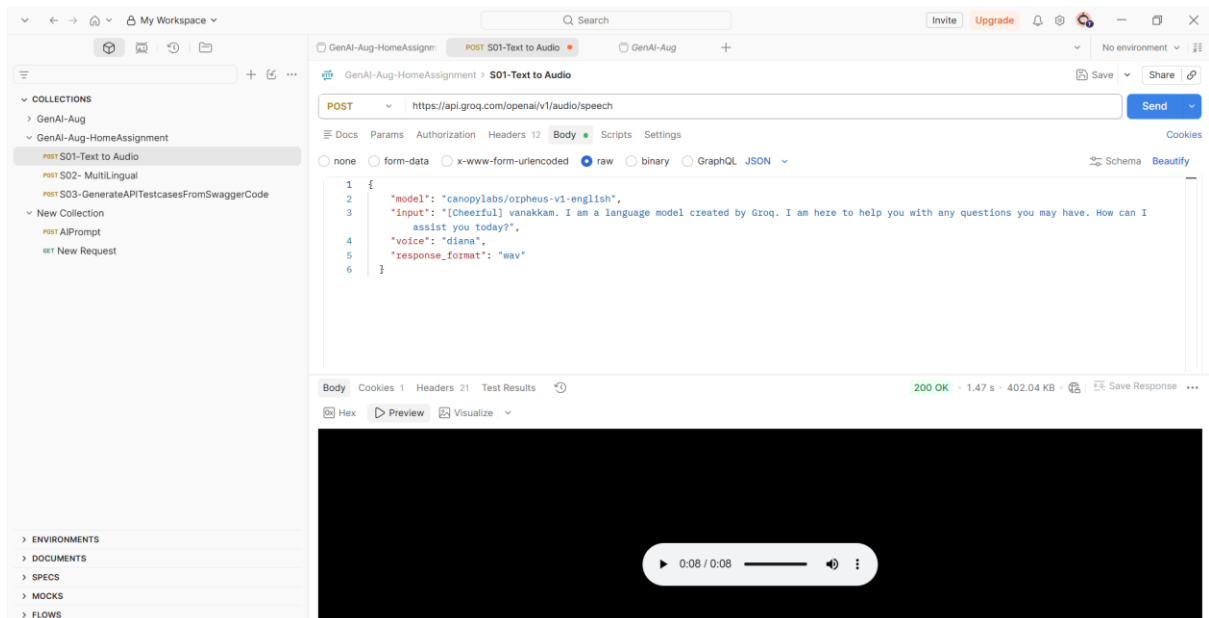


Week2_Day2_Assignment

Home Assignment:

1. Import the provided Postman collection.
2. Practice all **3 requests** available in the collection.
3. Take screenshots of the successful execution of each request.
4. Combine the screenshots into a **single PDF**.
5. Push the PDF to your **GitHub repository**.
6. Share the **GitHub repository link** in the **Assignment Tracker**.





```
"object": "chat.completion",
```

```
"created": 1787499224,

"model": "openai/gpt-oss-120b",

"choices": [

  {

    "index": 0,

    "message": {

      "role": "assistant",

      "content": "import io.restassured.RestAssured;\nimport
io.restassured.http.ContentType;\nimport io.restassured.response.Response;\nimport
org.testng.annotations.BeforeClass;\nimport org.testng.annotations.Test;\nimport
java.util.HashMap;\nimport java.util.Map;\nimport static io.restassured.RestAssured.given;\nimport
static org.hamcrest.Matchers.*;\n\npublic class PetApiTests {\n\n  private static long petId;\n
private static final String PET_NAME_INITIAL = \"Fluffy\";\n  private static final String
PET_NAME_UPDATED = \"FluffyUpdated\";\n\n  @BeforeClass\n  public void setUp() {\n
RestAssured.baseURI = \"https://petstore.swagger.io/v2\";\n    petId =
System.currentTimeMillis();\n  }\n\n  @Test\n  public void createPet() {\n    Map<String,
Object> pet = new HashMap<>();\n    pet.put(\"id\", petId);\n    pet.put(\"name\",
PET_NAME_INITIAL);\n    pet.put(\"status\", \"available\");\n\n    Map<String, String> category =
new HashMap<>();\n    category.put(\"id\", \"0\");\n    category.put(\"name\", \"Dogs\");\n
pet.put(\"category\", category);\n\n    Map<String, String> tag = new HashMap<>();\n
tag.put(\"id\", \"0\");\n    tag.put(\"name\", \"tag1\");\n    pet.put(\"tags\", new
Map[] {tag});\n\n    given()\n      .contentType(ContentType.JSON)\n      .body(pet)\n    .when()\n      .post(\"/pet/\")\n      .then()\n      .statusCode(200)\n      .body(\"id\",
equalTo((int) petId))\n      .body(\"name\", equalTo(PET_NAME_INITIAL))\n      .body(\"status\",
equalTo(\"available\"));\n  }\n\n  @Test(dependsOnMethods = {\"createPet\"})\n
public void getPetById() {\n    Response response = given()\n      .pathParam(\"petId\", petId)\n
.when()\n      .get(\"/pet/{petId}\")\n      .then()\n      .statusCode(200)\n      .body(\"id\",
equalTo((int) petId))\n      .body(\"name\", equalTo(PET_NAME_INITIAL))\n      .body(\"status\",
equalTo(\"available\"))\n      .extract().response();\n\n    // Additional optional
assertion using TestNG\n    org.testng.Assert.assertEquals(response.path(\"name\"),
PET_NAME_INITIAL);\n  }\n\n  @Test(dependsOnMethods = {\"getPetById\"})\n  public void
updatePet() {\n    Map<String, Object> updatedPet = new HashMap<>();\n
updatedPet.put(\"id\", petId);\n    updatedPet.put(\"name\", PET_NAME_UPDATED);\n
updatedPet.put(\"status\", \"pending\");\n\n    Map<String, String> category = new
HashMap<>();\n    category.put(\"id\", \"0\");\n    category.put(\"name\", \"Dogs\");\n
updatedPet.put(\"category\", category);\n\n    Map<String, String> tag = new HashMap<>();\n
tag.put(\"id\", \"0\");\n    tag.put(\"name\", \"tag1\");\n    updatedPet.put(\"tags\", new
Map[] {tag});\n\n    given()\n      .contentType(ContentType.JSON)\n      .body(updatedPet)\n
.when()\n      .put(\"/pet/\")\n      .then()\n      .statusCode(200)\n      .body(\"id\",
equalTo((int) petId))\n      .body(\"name\", equalTo(PET_NAME_UPDATED))\n      .body(\"status\",
equalTo(\"pending\"));\n  }\n\n  @Test(dependsOnMethods = {\"updatePet\"})\n
public void deletePet() {\n    given()\n      .pathParam(\"petId\", petId)\n      .when()\n
.delete(\"/pet/{petId}\")\n      .then()\n      .statusCode(200)\n      .body(\"message\",
equalTo(String.valueOf(petId))); \n  }\n}
```

"reasoning": "We need to output Java code, no markdown. Include imports, class, @BeforeClass, test methods using RestAssured, TestNG. Use reusable base URI. Use meaningful names. Should have request body creation, response validation with status code and important fields. Provide code that compiles.\n\nLet's write a class PetApiTests.\n\nSetup base URI in @BeforeClass using RestAssured.baseURI = \"https://petstore.swagger.io/v2\". Use static import of given/when/then etc.\n\nNeed createPet test: POST /pet with body. Use a petId generated maybe random long. Use Map or POJO? Simpler to use a JSON string or Map. Use a static variable petId to share across tests.\n\nTest getPetById uses GET /pet/{petId}, asserts status 200 and fields name etc.\n\nUpdatePet uses PUT /pet with body containing same petId but different name/status. Assert.\n\nDeletePet uses DELETE /pet/{petId}, assert status 200.\n\nMake sure to use TestNG annotations @Test(dependsOnMethods) for ordering.\n\nUse assertions: import static org.hamcrest.Matchers.*; and also org.testng.Assert.\n\nNow produce code."

```

    },
    "logprobs": null,
    "finish_reason": "stop"
  }
},
"usage": {
  "queue_time": 0.791410779,
  "prompt_tokens": 396,
  "prompt_time": 0.016390392,
  "completion_tokens": 1041,
  "completion_time": 2.170680978,
  "total_tokens": 1437,
  "total_time": 2.18707137,
  "completion_tokens_details": {
    "reasoning_tokens": 243
  }
},
"usage_breakdown": null,
"system_fingerprint": "fp_c868cf1eaa",
"x_groq": {
  "id": "req_01m0qm349tex6v30bjs4pt5btn",
  "seed": 1760454393
},

```

```
"service_tier": "on_demand"
```

```
}
```